Exercise 3: Arrays and Structs

Discussion: Tuesday, November 3rd

Problem 3.1: Arrays

Write your functions into the file `Exercise1_Arrays/Arrays.cpp`. Use `main.cpp` to test your functions. Feel free to take a look at the file `Exercise1_Arrays/Arrays.h` if you need help with creating the function signatures.

a) Given an array `grades` and its length `amount_of_grades`. Write a function `average` that takes a pointer to a `double[]` array and the number of elements (length) of the array as an `int` as parameters. The function should calculate the arithmetic mean of the array entries and return it as a `double` value.

   Calculation of the arithmetic mean:

$$\text{mean} = \frac{1}{n} \sum_{k=1}^{n} a_k$$

   Example:

$$\frac{4+3+2+1}{4} = 2.5$$

b) Implement a function `contains` that checks whether a given value is contained in an array. Since we are working with `double` values here, we cannot use a direct `==` comparison. Therefore, proceed in two steps and first implement a helper function:

   i) The `compare` function: This function receives two `double` values as parameters and should return `true` if the two values differ less than 0.00001; otherwise, the function should return `false`.

   For example, for the inputs 5.0 and 4.9:

$$\begin{aligned} |5.0 - 4.9| &< 0.00001 \\ \Leftrightarrow 0.1 &< 0.00001 \\ \Longrightarrow \text{false} & \end{aligned}$$

   ii) The `contains` function: Write a function `contains` that receives a pointer to a `double[]` array, its length as an `int`, as well as a `double` target value. The function should return the index of the target value if it is found in the array. If this is not the case, the function returns -1. Be sure to use the previously implemented `compare` function for the value comparison.

c) Briefly explain (in your own words) why it is not a good idea to compare two floating-point numbers using `==`. Also explain why we only pass a pointer to the array to the functions rather than the array itself.

d) Compare your implementation with a binary search (`contains_binary_search`):

- Do both functions do exactly the same thing?
- How many elements are checked on average or in the worst case before the target is found?
- What condition must necessarily be fulfilled for the array so that binary search yields correct results?

Hint 1: You can find corresponding explanations, for example, using the term "binary search" for an internet research. You can also ask an AI chatbot of your choice. Formulate the explanations in your own words.

Hint 2: Practice formulating explanations in your own words. It will be helpful for the exam ;-)

Problem 3.2: Arrays

Implementation Note: Please write your code for this exercise inside the `Exercise2_Structs/Structs.h` file. You can test your implementation inside `main.cpp` as usual.

a) Struct Declaration and Instantiation
Declare a `struct` named `Pizza` containing two `float` member variables:

- `diameter` (the diameter of the pizza in cm)
- `price` (the price of the pizza in Euros)

Inside your `main()` function, instantiate two `Pizza` objects using the values of your favorite pizzas (or custom example values).

b) Calculating Price per Square Centimeter
Implement the member function `price_per_cm2`, which calculates and returns the price per square centimeter of the pizza as a `float`.

c) Calculating Yield for Five Euros
Implement the member function `five_euro_get_you_x_cm2`. This method must internally utilize your `price_per_cm2` function to compute the total area (in cm^2) obtainable for 5.00 Euros. Return this value as a `float`.

Testing in `main()`

Test both methods within `main()` using your instantiated pizza objects to determine which pizza offers the best value for money!